QuickJS与V8：字节码和属性访问分析

马云霄 | 截至2026-10-17：本周做小程序对照与原 Richards 属性分析。

先看同一轮求和怎样执行

相同 JavaScript

```
function wbNumericLoop(n) {
  var sum = 0;
  for (var i=0; i<n; i++) {
    sum += i;
  }
  return sum;
}
```

n=4096；返回8386560。

整数结果，未溢出 int32。

QuickJS 摘录

```
get_loc1 1: i
add_loc 0: sum
inc_loc 1: i
goto8 4
```

add_loc 更新局部sum，
并弹栈；inc_loc 更新i。

V8 Ignition 摘录

```
Ldar r1
Add r0, EmbeddedFeedback[None]
Mov r0, r2
Star0
Ldar r1
Inc EmbeddedFeedback[None]
Star1
JumpLoop [21], [0], FBV[0]
```

r0=sum，r1=i；Mov复制旧sum，Star0写新sum。

Median（μs/完整调用）：QJS **43.782**；V8 54.839；V8/QJS 1.253。

每调用含4096轮；每引擎30个新进程。虚拟槽不是CPU寄存器。
本例QJS更快；指令条数/字节数不等于成本，也不能据此断言栈式更优。

来源：S1 numeric_loop.js、真实dump、summary.csv。S1/S2/S3完整路径见讲稿。
WSL2 Ubuntu24.04.2/x86_64；QuickJS 2026-06-04；V8 15.6.21（d8）；--max-opt=0 --no-lazy。
内部单调计时包含首次调用；无计时前workload预跑。点估计，不是单条指令成本。

四组调用方式：一层原型够不够解释差距?

共同计算：i=0..4095，bias=3，累加 i+bias，结果8398848。

组别	循环中的主要表达式	QJS / µs	V8 / µs	V8/QJS
直接计算	`sum += i + obj.bias`	67.132	60.388	0.900
普通调用	`sum += stepPlain(obj,i)`	159.969	116.395	0.728
own 方法	`sum += obj.step(i)`	217.156	120.905	**0.557**
prototype 方法	`sum += obj.step(i)`	225.086	125.348	**0.557**

中位时间：µs/完整目标调用，含4096轮及摊薄driver；每组每引擎30个新进程，共240样本。

```
function stepPlain(obj, x) {
  return x + obj.bias;
}
function stepMethod(x) {
  return x + this.bias;
}
```

own/prototype比值都约0.557。

同一循环和方法，只在计时外改变对象建立方式。
一层原型不足以解释差距；参数、receiver、shape/反馈也会变，不能相减当纯调用成本。
本页含obj.bias和两次加法，不是第1页求和。

来源：S2 四组cases、summary.csv、raw/timing.csv。对象建立在计时外，方法读取保留在循环内。
WSL2 Ubuntu24.04.2/x86_64；QuickJS 2026-06-04；V8 15.6.21（d8）；--max-opt=0 --no-lazy。
内部单调计时包含首次调用；无计时前workload预跑。点估计，不是单次操作成本。

原始 Richards：查找区域样本落在哪里?

Richards 是模拟任务调度的 JavaScript 基准。
本轮使用未修改的原算法、方法读取和断言。

Scheduler.schedule 摘录

此处 this 是 Scheduler。

```
this.currentTcb = this.currentTcb.run();
```

TaskControlBlock.run 摘录

进入此函数后，this 是当前 TCB。

```
return this.task.run(packet);
```

currentTcb 指向TCB；TCB.task指向具体任务。
取得对象、读方法、调用函数不是同一步。

属性	样本	占1229
currentTcb	280	22.8%
state	204	16.6%
两处run	79	6.4%

3个诊断进程，共5393个内部样本；
1229个查找区域样本已映射。
按属性聚合，包含多个读/写位置。

分母是1229，不是全程序耗时。普通状态字段值得继续看，尚未找到总体瓶颈。

方法说明：189个活跃opcode位置；N1/N2倍增核对，计数/采样分开；另39次helper探查无具体site。

来源：S3 原richards.js、site_samples.csv、sampling_summary.json、count_audit.json。
WSL2 Ubuntu24.04.2/x86_64；QuickJS 2026-06-04；V8 15.6.21（d8）；--max-opt=0 --no-lazy。
诊断数据，不作正式性能结果；采样占比不是精确self-time。

选定 this.state：按源码语义分组读字节码

原函数 TaskControlBlock.prototype.isHeldOrSuspended：

```
return (this.state & STATE_HELD) != 0 || (this.state == STATE_SUSPENDED);
```

调用者schedule的this是Scheduler；进入本函数后this是当前TCB。

QuickJS

```
0 push_this
 1 put_loc0 0: this
 2 get_loc0 0: this          ①取this
 3 get_field state           读state

 8 get_var 0: STATE_HELD
11 and
13 neq                       ②与/比较
15 if_true8 28               短路

18 get_loc0 0: this
19 get_field state           ③再读取
```

V8 Ignition

```
0 GetNamedProperty <this>, [0:"state"], FBV[0]
 4 Star0

 5 LdaGlobal [1:"STATE_HELD"], FBV[2]
 8 BitwiseAnd r0, EmbeddedFeedback[None]
13 TestEqual r0, EmbeddedFeedback[None]
16 LogicalNot
17 JumpIfTrue [13] (0x1eef0100166e @ 30)

19 GetNamedProperty <this>, [0:"state"], FBV[4]
```

①offset 0–4；②5–17；③19。r0是虚拟frame槽。

非连续真实摘录，完整序列见附录；左侧为真会短路。offset不是native PC，虚拟槽不是CPU寄存器。

来源：S3 richards.js:309–311；两引擎 isHeldOrSuspended 的真实最终dump与decoded.json。
WSL2 Ubuntu24.04.2/x86_64；QuickJS 2026-06-04；V8 15.6.21（d8）；--max-opt=0 --no-lazy。
诊断数据，不作正式性能结果；正文省略push_0、dup等，不能视为连续可执行序列。

这一处实际知道了什么?

QJS：第一次state读取，offset 3

每 Run **10671次**，全部own数据属性命中。
每次一次自身探查；结果为整数。

原型命中0；通用fallback 0。
不等于这次读取没有成本。

仍有shape/hash查找、描述符检查、
取值及必要的receiver引用处理。

整数state本身不做对象引用计数；
不要把receiver与字段值混为一谈。

V8：同进程字节码与IC转换日志

```
offset 0: FBV[0] 0 → 1
offset 19: FBV[4] 0 → 1
receiver map: 0x189b01023ef1
```

两处有独立反馈槽，观察到相同map。
转换日志不是逐次命中记录；
IC命中率仍 UNKNOWN。

机器码有map检查/字段读取路径，
不证明该site每次走此路径。
精确源码/构建与制品对应仍有缺口。

待验证解释：QJS重复自身查找；V8可能复用稳定字段反馈。

目前不是已证明的性能原因；尚未测得各步骤的时间，也未证明该site的逐次快路径。

来源：S3 counts_N1.sites.csv、v8_selected_IC_transitions.csv、实际GetNamedProperty反汇编。
WSL2 Ubuntu24.04.2/x86_64；QuickJS 2026-06-04；V8 15.6.21（d8）；--max-opt=0 --no-lazy。
诊断数据，不作正式性能结果；状态转换与handler分支存在不等于逐次命中证据。

本周能说什么，还有什么不懂?

目前能说

操作不同，快慢方向可能不同；不把小程序拼成总体引擎排名。

多一层原型不足以解释方法案例的跨引擎差距。

原 Richards 的普通own字段值得深入；已选出可具体分析的state位置。

两个直接限制：诊断数据，不是正式性能结果

无信号N512小对照：冻结版median1718ms，诊断开启版2223ms，高约**29%**。

不同批次的点估计，不是精确插桩成本；时间接近也不能证明没有扰动。

两次state读取每Run为10671/10115次，查找区域样本却为**85/9**。

请求1ms、实际约10ms；采样相位和插桩会影响分布，不能视为精确耗时排名。

想与老师讨论

下一步先确认V8该位置的实际快路径，还是先降低采样扰动、复核位置分布?

来源：S1/S2正式汇总；S3 perturbation.json、site_samples.csv、counts_N1.sites.csv。

WSL2 Ubuntu24.04.2/x86_64；QuickJS 2026-06-04；V8 15.6.21（d8）；--max-opt=0 --no-lazy。

诊断数据，不作正式性能结果；本周未实现VM优化，未恢复方法外提A/B。

附录A：QuickJS 完整最终指令序列

原 TaskControlBlock.prototype.isHeldOrSuspended；16条/29B，局部0=this，栈深2。
补齐offset，保留全部指令和操作数；去掉源码注释与debug表。

```
0 push_this
 1 put_loc0 0: this
 2 get_loc0 0: this
 3 get_field state
 8 get_var 0: STATE_HELD
11 and
12 push_0 0
13 neq
14 dup
15 if_true8 28
17 drop
18 get_loc0 0: this
19 get_field state
24 get_var 1: STATE_SUSPENDED
27 eq
28 return
```

与正文第4页核对

push_0、dup、drop在正文省略；
完整序列在此保留。

来自独立打印诊断构建。
不能把其native代码或耗时
视为冻结Release版的性能。

静态条数和编码长度不是执行时间。

来源：S3 bytecode/quickjs/TaskControlBlock.prototype.isHeldOrSuspended.txt 与 decoded.json。
WSL2 Ubuntu24.04.2/x86_64；QuickJS 2026-06-04；V8 15.6.21（d8）；--max-opt=0 --no-lazy。
诊断数据，不作正式性能结果；最终执行字节码，不是中间表示。

附录B：Ignition 完整最终指令序列

原 TaskControlBlock.prototype.isHeldOrSuspended；14条/31B，虚拟寄存器1，frame 8B。
去掉左侧地址/编码/调试列；以下指令与操作数完整，保留实际jump目标地址。

```
0 GetNamedProperty <this>, [0:"state"], FBV[0]
 4 Star0
 5 LdaGlobal [1:"STATE_HELD"], FBV[2]
 8 BitwiseAnd r0, EmbeddedFeedback[None]
11 Star0
12 LdaZero
13 TestEqual r0, EmbeddedFeedback[None]
16 LogicalNot
17 JumpIfTrue [13] (0x1eef0100166e @ 30)
19 GetNamedProperty <this>, [0:"state"], FBV[4]
23 Star0
24 LdaGlobal [2:"STATE_SUSPENDED"], FBV[6]
27 TestEqual r0, EmbeddedFeedback[None]
30 Return
```

QJS 16条/29B，V8 14条/31B：条数、编码长度与native分发次数不是同一指标，也不等于执行成本。

来源：S3 bytecode/v8/TaskControlBlock.isHeldOrSuspended.txt 与 decoded.json；IC见同进程日志。
WSL2 Ubuntu24.04.2/x86_64；QuickJS 2026-06-04；V8 15.6.21（d8）；--max-opt=0 --no-lazy。
诊断数据，不作正式性能结果；r0是虚拟槽，不是CPU物理寄存器。